

3

プログラミング概論I (オンライン授業)



担当: 中山 功一 (7号館3階308室)

メールアドレス: knakayama@is.saga-u.ac.jp

188

オープニングテスト

前回, 習ったことを復習するために,
オープニングテストを行います

問題ごとに動画を止めて, 答えをメモしましょう.

最後に, 答えを動画で表示しますので,
自分が理解しているか, 確認しましょう.

オープニング テスト

- 下の文を実行した後のiの値を答えなさい

問1: $i = 10 \% 3 - 1;$

問2: $i = 2 + 1 * 2;$

問3: $i = 2 / 2 * 2;$

問4: $i = 3 \% 2 * 2;$

問5: $i = 4 \% 5;$

問6: $i = 2 * (2 + 1);$

<選択肢>

① 1 ⑥ 6

② 2 ⑦ 7

③ 3 ⑧ 8

④ 4 ⑨ 9

⑤ 5 ⑩ 0

190

クロージング テスト

問7~10: 次のプログラムの出力を答えなさい

```
int a = 2, b = 4, c;  
c = 1 + a / 2;  
cout << c;    //問7  
b -= a + b % c;  
cout << b;    //問8  
a /= c;  
cout << a;    //問9  
a++;  
cout << a;    //問10
```

<選択肢>

① 1 ⑥ 6

② 2 ⑦ 7

③ 3 ⑧ 8

④ 4 ⑨ 9

⑤ 5 ⑩ 0

191

答え合わせ

全ての答えをメモしましたか?

解答

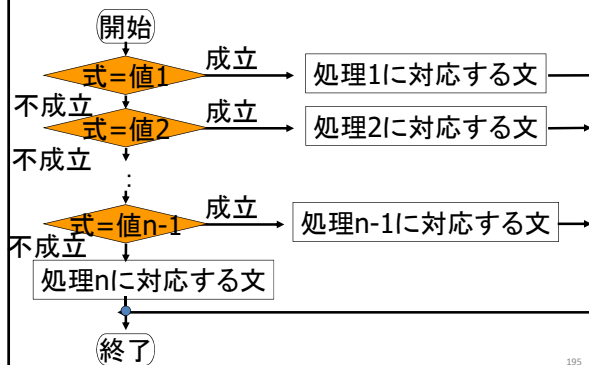
- 【1】 ① 1 - 1
- 【2】 ④ 2 + 2
- 【3】 ② 1 * 2
- 【4】 ② 1 * 2
- 【5】 ④ 4 / 5 = 0 あまり 4
- 【6】 ⑥ 2 * 3
- 【7】 ② 1 + 1
- 【8】 ② 4 -= 2 + 0
- 【9】 ① 2 /= 2
- 【10】 ② 1++

switch文のアルゴリズム

0. 式の値によって場合分けする
1. 式が値1と一致すれば、そこから処理を実行する
 - 1-1 処理1
 - break;で処理をやめる
2. 式が値2と一致すれば、そこから処理を実行する
 - 2-1 処理2
 - break;で処理をやめる
 - :
- n. 式がどの値とも一致しないならば、そこから処理を実行する
 - n-1 処理n

194

switch文の動作



195

switch文の文法

```

// 式の値によって場合分け
switch (式) {
    // 式が値1と一致すれば、
    // 以下の処理を実行
    case 値1:
        // 処理1
        break;

    // 式が値2と一致すれば、
    // 以下の処理を実行
    case 値2:
        // 処理2
        break;

    // 式が値3と一致すれば、
    // 以下の処理を実行
    case 値3:
        // 処理3
        break;

    // 式がどの値とも一致しないならば、以下の処理を実行
    default:
        // 処理n
        break;
}
  
```

switch文の文法

```

switch (式) {
    case 値1:
        // 処理1
        break;
    case 値2:
        // 処理2
        break;

    default:
        // 処理n
        break;
}
  
```

多岐条件文で書ける

switch文の文法

```

switch (式) {
    case 値1:
        // 処理1
        break;
    case 値2:
        // 処理2
        break;

    :
    default:
        // 処理n
        break;
}

if (式 == 値1) {
    // 処理1
} else if (式 == 値2) {
    // 処理2
} else if (式 == 値3) {
    // 処理3
} ...
...
} else {
    // 処理n
}
  
```

多岐条件文で書ける

例

入力された値が
1なら「一」、2なら「二」、
それ以外なら「それ以外」と表示

```
cin >> suuji;
switch(suuji){
  case 1:
    cout << "一";
    break;
  case 2:
    cout << "二";
    break;
  default:
    cout << "それ以外";
    break;
}
```

199

注意点

- 「;」と「:」を間違えないように
– 「case 値」の後ろは「:」
- 「break;」を忘れると、次のcase節も実行される
break文のないswitch文は使用しない
(例外: 文が空の場合)
- defaultラベルは、予期しない場合を検出するために必ず**最後**におく

200

1, 2, 3をそれぞれ入力する. 画面出力は?

```
cin >> suuji;
switch(suuji){
  case 1:
    cout << "一";
    break;
  case 2:
    cout << "二";
    break;
  default:
    cout << "それ以外";
    break;
}
```

- 1を入力
「一」
- 2を入力
「二それ以外」
- 3を入力
「それ以外」

201

多岐条件文

- 複数(3つ以上)の場合に分けて処理する
- switch文では記述できない場合に使用
– switch文は等号比較のみ可能
– switch文で書ける場合には、switch文を使う

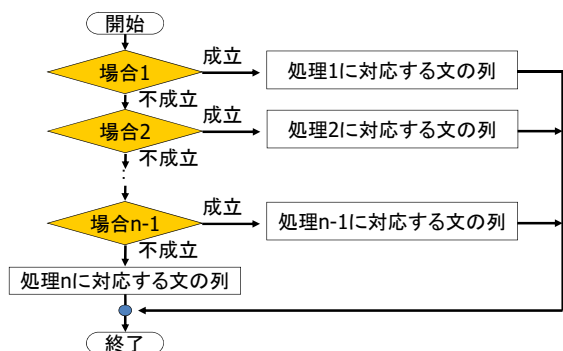
202

多岐条件文のアルゴリズム

1. **場合1**が成立すれば、以下の処理を実行する
1-1. 処理1
2. **場合2**が成立すれば、以下の処理を実行する
2-1. 処理2
3. **場合3**が成立すれば、以下の処理を実行する
3-1. 処理3
⋮
- n. **どの場合も成立しない**ならば、以下の処理を実行する
n-1. 処理n

203

多岐条件文の動作



204

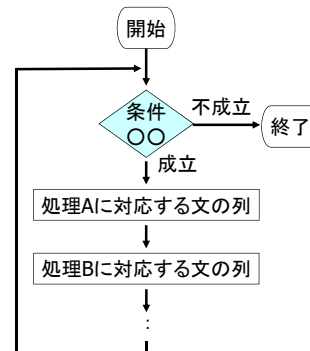
繰り返し(ループ)

while文(繰り返し)

- 処理を繰り返す(繰り返し回数は未知)

```
// 条件○○が成り立つ限り処理A,Bを繰り返す
while ( ○○ ) {
    処理A // 処理A
    処理B // 処理B
}
```

while文の動作



```
while( i<N ) {
    処理A;
    処理B;
}
```

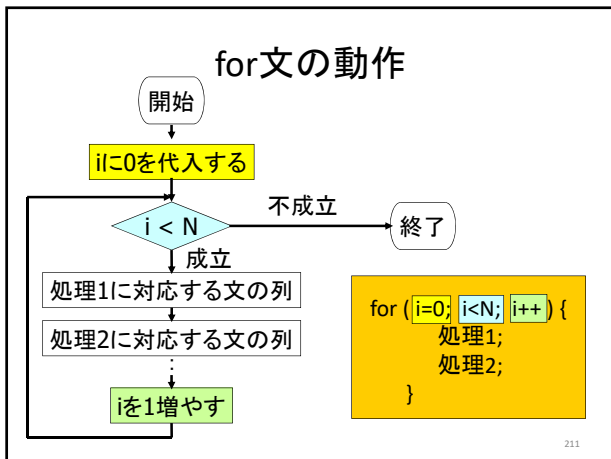
for文(繰り返し)

- 指定された回数だけ処理を繰り返す
- // 変数iの値を0からN-1まで変更しながら
// 以下の処理を繰り返す

```
int i;
for (i=0; i<N; i++) {
    処理1;
    処理2;
}
```

または

```
for (int i =0; i<N; i++)
{
    処理1;
    処理2;
}
```



for文の記述

0からN-1までN回繰り返す	for (i = 0; i < N; i++)
aからbまで繰り返す	for (i = a; i <= b; i++)
Nから1まで降順で繰り返す	for (i = N; i > 0; i--)
bからaまで降順で繰り返す	for (i = b; i >= a; i--)
2ずつ増えながら繰り返す	for (i = 0; i < N; i += 2)
2倍しながら繰り返す	for (i = 1; i < N; i *= 2)

212

配列の範囲for文

```

int array[5] = { 1, 2, 3, 4, 5 };
//通常のfor文
for (int i = 0; i < 5; i++) {
    cout << array[i] << endl;
}

//範囲for文
for (int i : array) {
    cout << i << endl;
}

```

出力はどちらも同じ

1
2
3
4
5

213

配列の範囲for文

```

//範囲for文
for (int i : array) {
    cout << i << endl;
}

```

配列arrayのすべての要素が
順番にループ変数 i に代入される

※ 範囲を限定する場合は使えない
(すべての要素にアクセスする場合に使う)

214

215

216

アルゴリズム

未成年飲酒犯の捕まえ方

- (__) 名前を記録する
- (__) 未成年であるかを確認する
- (__) お酒を飲んでいるかを確認する
- (4) 逮捕！

課題: 効率の良い順番を書け

218

正解は・・・？

未成年飲酒犯の見つけ方

路上検問の場合(飲んでいる人が少ない)

- (3) 名前を記録する
- (2) 未成年であるかを確認する
- (1) **お酒を飲んでいるかを確認する**
- (4) 逮捕！

220

未成年飲酒犯の見つけ方

居酒屋の場合(飲んでいる人が多い)

- (3) 名前を記録する
- (1) **未成年であるかを確認する**
- (2) お酒を飲んでいるかを確認する
- (4) 逮捕！

無駄な作業をしない手順が良い

221

222

カップラーメンの作り方

- (1) お湯をたっぷり沸かす
- (2) カップラーメンの蓋をあける
- (3) 沸いたお湯を入れる
- (4) ふたを閉める
- (5) 3分待つ
- (6) 完成！

課題: 10人で食べるので,
10個のカップラーメンをつくろう

223

ダメな回答例

```
for(10回繰り返す){  
  (1) お湯をたっぷり沸かす  
  (2) カップラーメンの蓋をあける  
  (3) 沸いたお湯を入れる  
  (4) ふたを閉める  
  (5) 3分待つ  
}  
のびのび麺が完成... (≫_<<)
```

224

ちょっと残念な回答例

```
(1) お湯をたっぷり沸かす  
for(10回繰り返す){  
  (2) カップラーメンの蓋をあける  
  (3) 沸いたお湯を入れる  
  (4) ふたを閉める  
}  
(5) 3分待つ  
  ちょっと冷めた麺が完成...
```

225

正しい回答例

```
(1) お湯をたっぷり沸かす  
for(10回繰り返す){  
  (2) カップラーメンの蓋をあける  
}  
for(10回繰り返す){  
  (3) 沸いたお湯を入れる  
  (4) ふたを閉める  
}  
(5) 3分待つ  
  美味しい麺が完成! ( $\geq v \leq$ )
```

良いアルゴリズムを考えよう

226

アルゴリズムの基本制御構造

- アルゴリズムは、基本制御構造のみを用いて記述する(構造化プログラム)
 1. **ブロック構造(順次実行)**
 - 記述したとおりの順番で実行される命令群
 2. **条件判定**
 - 条件判断によって選択的に実行される命令群
 3. **繰り返し**
 - 繰り返し条件によって繰り返される命令

<参考: 掛下先生のページ>

<http://www.cs.is.saga-u.ac.jp/syllabus/GuideLine/algorithm.html>

アルゴリズムの記法(1. **ブロック構造**)

- 順番に実行される操作の系列を記述

```
(1) 操作1  
(2) 操作2  
(3) 操作3  
  .  
  .  
(n) 操作n
```

228

アルゴリズムの記法(2.条件判定)

- 条件を用いて場合分けする場合に使用
- if文に対応

- (1) 条件〇〇が成立するならば、処理□□を実行
(2) そうでなければ、処理△△を実行する

```
if(〇〇) {  
    処理□□;  
} else {  
    処理△△;  
}
```

229

アルゴリズムの記法(3.繰り返し)

- 条件が成立するまで、処理を繰り返す
- for, while 文に対応

- (1) 条件〇〇が成立している限り、□□を実行する

```
while(〇〇) {  
    処理□□;  
}
```

```
for(int i = 1; i <= N; i++) {  
    処理△△;  
}
```

- (1) iを1からNまで増加させながら、△△を実行する

230

探索アルゴリズム

探索問題

- 配列(要素数N)に保存されている要素の中から、指定された値を持つものを探す
- 順次探索アルゴリズム (Linear Search)
(線形探索, 逐次探索とも呼ぶ)
 - アルゴリズムが単純
 - Nに比例する手間がかかる
- 二分探索アルゴリズム (Binary Search)
 - データが大小順に並べられている必要がある
 - log Nに比例する手間が良い
 - 順次探索アルゴリズムより高速

232

定数: N 30 配列の要素数

論理型変数: found探索フラグ、探索成功時にtrue

整数型変数: value探索する値

順次探索

- 探索フラグをfalseにする
- 探索する値を入力する
- 変数iの値を配列の先頭から末尾まで変えながら以下の処理を繰り返す
 - 探索する値とi番目の要素が一致したら、以下の処理を実行する
 - iを表示する
 - 探索フラグをtrueにする
- 探索フラグがfalseのままならば、「探索失敗」と表示する

233

順次探索

```
int i, flag=0;  
int array[10]={/*略*/};  
for(i = 0 ; i < 10 ; i++){  
    if(array[i]==3){  
        flag = 1;  
    }  
}  
if(flag == 1){  
    cout << "3を含む";  
}else{  
    cout << "3を含まず";  
}
```


順次探索(逐次探索)

- 配列要素を先頭から順に調べてゆく

9	4	1	8	3	7	10	5	6
---	---	---	---	---	---	----	---	---



```
for(i = 0 ; i < 10 ; i++){  
    if(array[i]==3){  
        flag = 1;  
    }  
}
```

順次探索(逐次探索)

- 配列要素を先頭から順に調べてゆく

9	4	1	8	3	7	10	5	6
---	---	---	---	---	---	----	---	---



3を調べる

```
for(i = 0 ; i < 10 ; i++){  
    if(array[i]==3){  
        flag = 1;  
    }  
}
```

順次探索(逐次探索)

- 配列要素を先頭から順に調べてゆく

9	4	1	8	3	7	10	5	6
---	---	---	---	---	---	----	---	---



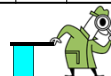
3を調べる

```
for(i = 0 ; i < 10 ; i++){  
    if(array[i]==3){  
        flag = 1;  
    }  
}
```

順次探索(逐次探索)

- 配列要素を先頭から順に調べてゆく

9	4	1	8	3	7	10	5	6
---	---	---	---	---	---	----	---	---



3を調べる

```
for(i = 0 ; i < 10 ; i++){  
    if(array[i]==3){  
        flag = 1;  
    }  
}
```

順次探索(逐次探索)

- 配列要素を先頭から順に調べてゆく

9	4	1	8	3	7	10	5	6
---	---	---	---	---	---	----	---	---



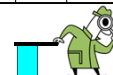
3を調べる

```
for(i = 0 ; i < 10 ; i++){  
    if(array[i]==3){  
        flag = 1;  
    }  
}
```

順次探索(逐次探索)

- 配列要素を先頭から順に調べてゆく

9	4	1	8	3	7	10	5	6
---	---	---	---	---	---	----	---	---



3を調べる

```
for(i = 0 ; i < 10 ; i++){  
    if(array[i]==3){  
        flag = 1;  
    }  
}
```

順次探索(逐次探索)

- 配列要素を先頭から順に調べてゆく

9	4	1	8	3	7	10	5	6
---	---	---	---	---	---	----	---	---



3を調べる

フラグを立てる

```
for(i = 0 ; i < 10 ; i++){
    if(array[i]==3){
        flag = 1;
    }
}
```

順次探索(逐次探索)

- 配列要素を先頭から順に調べてゆく

9	4	1	8	3	7	10	5	6
---	---	---	---	---	---	----	---	---



3を調べる

```
for(i = 0 ; i < 10 ; i++){
    if(array[i]==3){
        flag = 1;
    }
}
```

順次探索(逐次探索)

- 配列要素を先頭から順に調べてゆく

9	4	1	8	3	7	10	5	6
---	---	---	---	---	---	----	---	---



3を調べる

```
for(i = 0 ; i < 10 ; i++){
    if(array[i]==3){
        flag = 1;
    }
}
```

順次探索(逐次探索)

- 配列要素を先頭から順に調べてゆく

9	4	1	8	3	7	10	5	6
---	---	---	---	---	---	----	---	---



3を調べる

```
for(i = 0 ; i < 10 ; i++){
    if(array[i]==3){
        flag = 1;
    }
}
```

順次探索(逐次探索)

- 配列要素を先頭から順に調べてゆく

9	4	1	8	3	7	10	5	6
---	---	---	---	---	---	----	---	---



3を調べる

```
for(i = 0 ; i < 10 ; i++){
    if(array[i]==3){
        flag = 1;
    }
}
```

順次探索(逐次探索)

- 配列要素を先頭から順に調べてゆく

9	4	1	8	3	7	10	5	6
---	---	---	---	---	---	----	---	---



<3を調べる>

(Flag == 1) → 「3は含まれている！」

順次探索

```
int i, flag=0;
int array[10]={/*略*/};
for(i = 0 ; i < 10 ; i++){
    if(array[i]==3){
        flag = 1;
    }
}
if(flag == 1){
    cout << "3を含む";
}else{
    cout << "3を含まず";
}
```

問題

配列の何番目に
"3"があったかを
表示するプログラ
ムに改変しなさい

```
0: int i, flag=0;
1: int array[10]={/*略*/};
2: for(i = 0 ; i < 10 ; i++){
3:     if(array[i]==3){
4:         flag = 1;
5:     }
6: }
7: if(flag == 1){
8:     cout << "3を含む";
9: }else{
0:     cout << "3を含まず";
0: }
```

正解は・・・？

問題

配列の何番目に
"3"があったかを
表示するプログラ
ムに改変しなさい

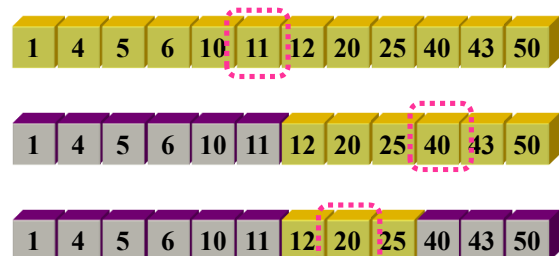
```
int i, flag= -1;
int array[10]={/*略*/};
for(i = 0 ; i < 10 ; i++){
    if(array[i]==3){
        flag = i;
    }
}
if(flag == -1){
    cout << "3を含まず";
}else{
    cout << flag << "番目";
}
```

二分探索の動きを理解しよう

- (1) 当てられた人は指示担当者です
作業担当者(中山)に指示して下さい
- (2) 私の目の前に昇順に並んだ
トランプが10枚あります
この中に「3」があるかを調べてください

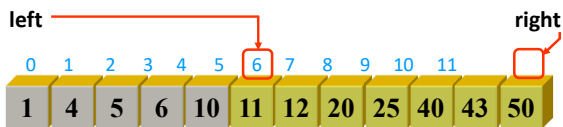
- ※ 指示担当者は、カードを見れません
- ※ 指示担当者は、3回しか指示できません
- ※ 作業担当者は、1回しか返事をしません

二分探索 (例: 昇順のデータから20を探す)



発見！！
(3回チェックしただけ)

二分探索



ポイント:

- チェックごとに, right-left の値が半分になる
- データ数がNの時, $\log N$ 回のチェックでOK (ループ4の繰り返しが $\log N$ 回)

253

例: 辞書の検索

Q: 広辞苑(約4,000ページ), 調べたい単語の掲載ページを調べるのに, 何ページ調べれば良いか?

A: 二分探索で調べる: 「調べる対象ページのうち, 真中のページを調べる」を繰り返す

4,000 $\Rightarrow$ 2,000 $\Rightarrow$ 1,000 $\Rightarrow$ 500 $\Rightarrow$ 250 $\Rightarrow$ 125
 $\Rightarrow$ 63 $\Rightarrow$ 32 $\Rightarrow$ 16 $\Rightarrow$ 8 $\Rightarrow$ 4 $\Rightarrow$ 2 $\Rightarrow$ 1

覚えよう!
 $2^{10} \div 1000$

(12回ほど)

$2^{12} \div 4000$

254

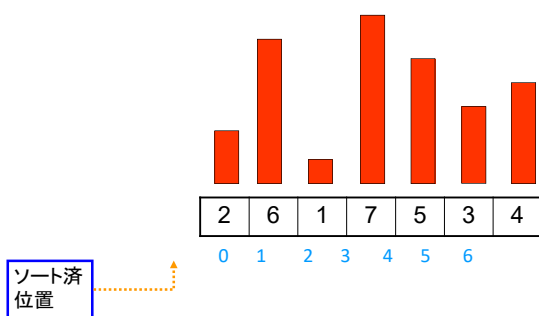
二つの探索アルゴリズムの違い

- 配列要素を何回チェックするか?
 - 1GHz (1ギガヘルツ)
 - 1秒間に10億回の命令を実行すると, 27個で良い
 - データが1億個あるとき
 - 1億個のデータを, 順番に調べる... 0.1[秒]
 - 1000人が同時に探索をかける
 - 合計100[秒] (1分40秒)
- 0.000000027[秒]

255

ソーティングアルゴリズム (※同じ数字はないものとする)

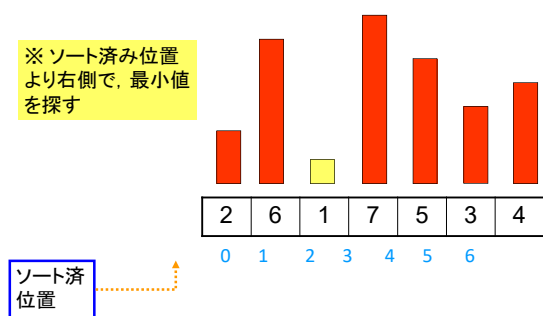
選択ソートの動き



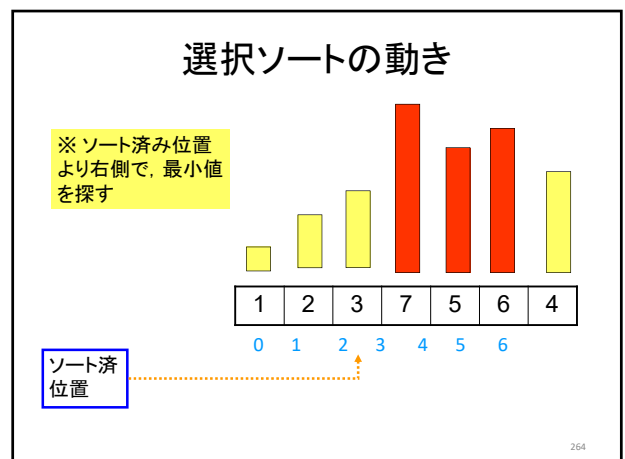
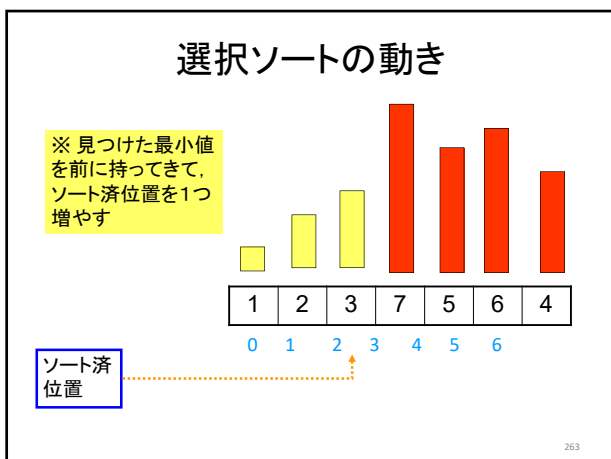
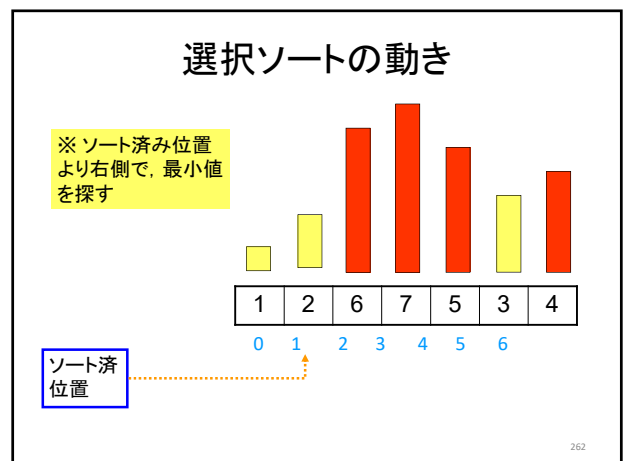
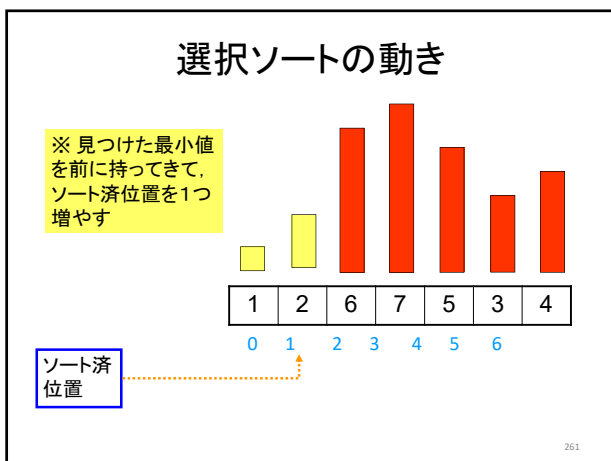
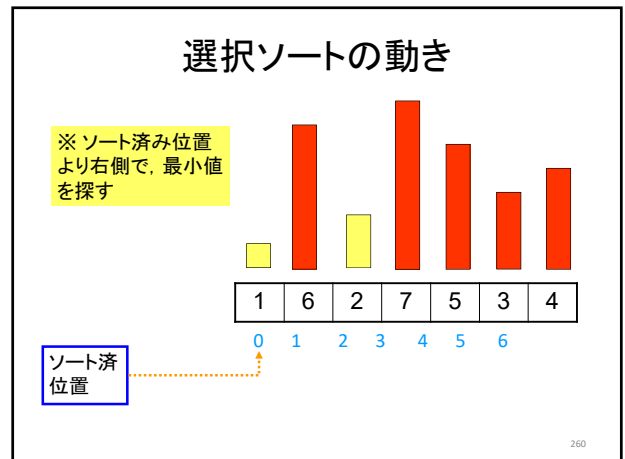
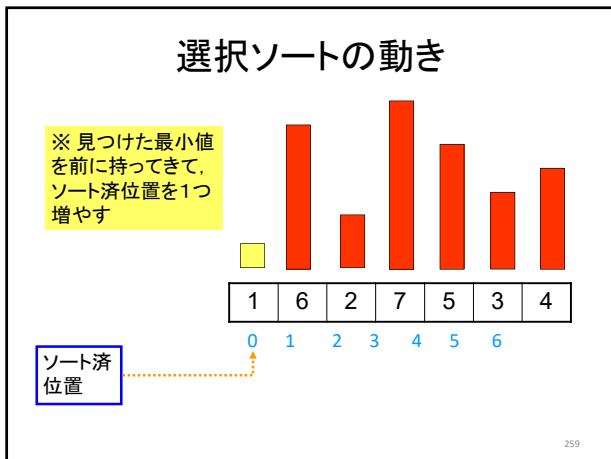
257

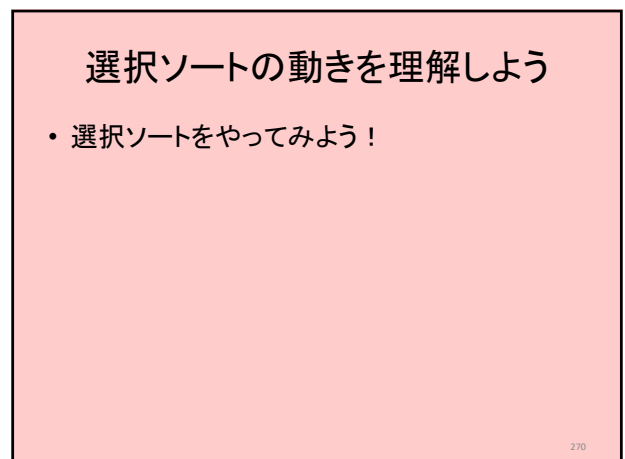
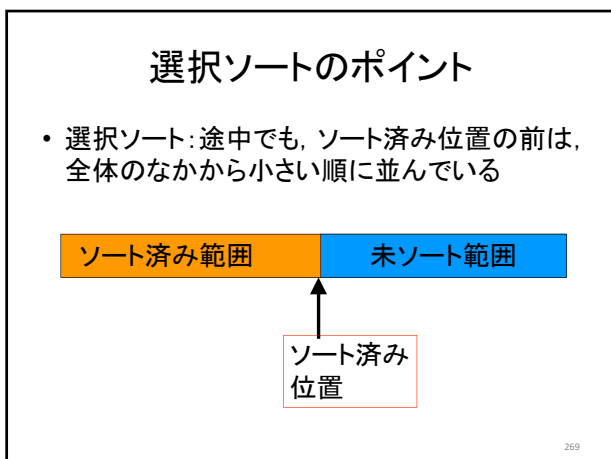
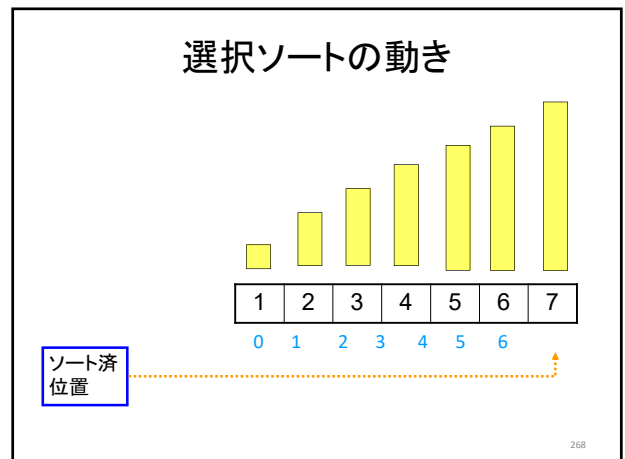
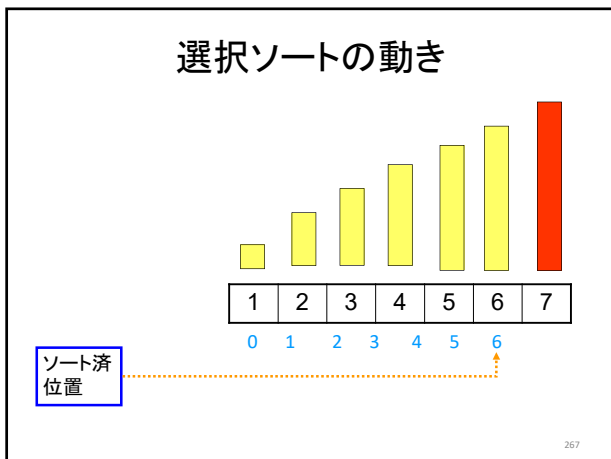
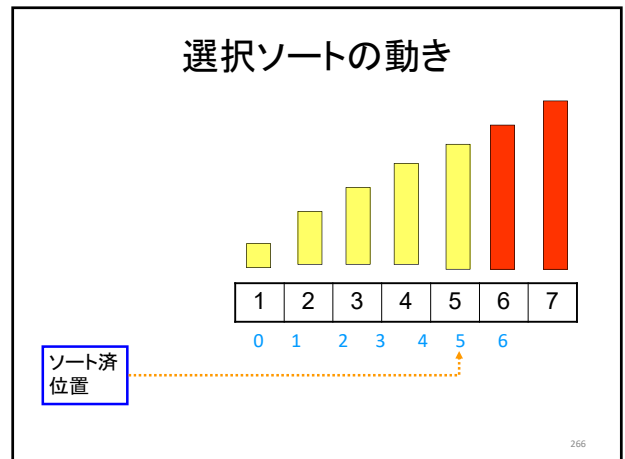
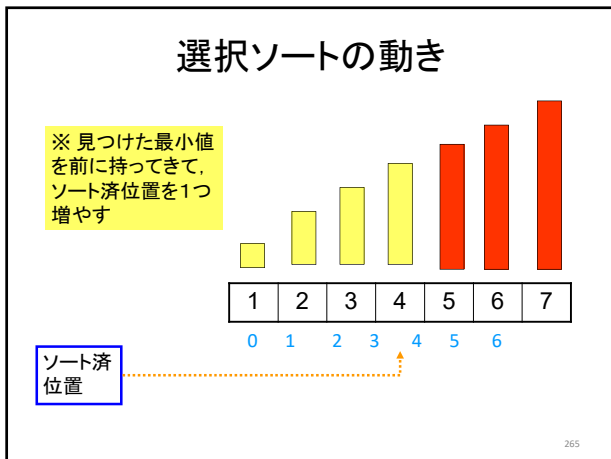
選択ソートの動き

※ ソート済み位置より右側で, 最小値を探す



258





選択ソートのアルゴリズム(概略)

- 1: 未ソート部分について以下の処理を行う
 - 1-1: 最も小さい値を選択する
 - 1-2: 選択した値と
未ソート部の左端を交換する
- 2: ソート済み部分を増やす
(= 未ソート部分を減らす)

271

選択ソートのアルゴリズム(詳細)

0. ソートする配列をdata[N]とする
1. ソート済み位置(s_pos)を配列先頭(0)の手前(-1)とする
2. ソート済み位置(s_pos)が配列末尾(N-1)になるまで
ソート済み位置を1ずつ増やしなが以下を繰り返す
 - 2-1. ソート済み位置+1(s_pos + 1)を仮の最小値とし、
その位置(s_pos + 1)を最小位置(min_pos)格納
 - 2-2. 現在位置(i)をソート済み位置+2から
末尾まで変えながら、以下の処理を繰り返す
 - 2-2-1. 現在位置(i)の要素(data[i])が
最小位置の要素(data[min_pos])より小さいなら
その位置(i)を最小値(min_pos)に代入する
 - 2-3. 最小値とソート済み位置+1の要素を交換する

272

```
int data[N];           //要素数Nの配列
int s_pos;             // ソート済み位置
int min_pos;          //最小値の位置
int i;                // 現在位置

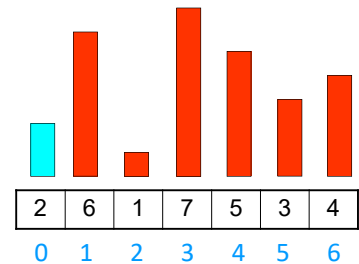
for (s_pos = -1; s_pos < N-1; s_pos++) {
    min_pos = s_pos+1; // 仮の最小位置を代入
    for (i = s_pos+2; i < N; i++) {
        if (data[i] < data[min_pos]) { // もし最小値なら
            min_pos = i;              // 最小位置の変更
        }
    }
    int t = data[min_pos]; // 要素の交換
    data[min_pos] = data[s_pos+1];
    data[s_pos+1] = t;
}
```

コード例

273

挿入ソートの動き

挿入要素

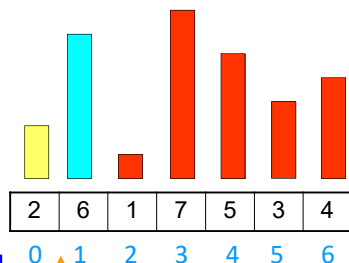


ソート済
位置

274

挿入ソートの動き

挿入要素

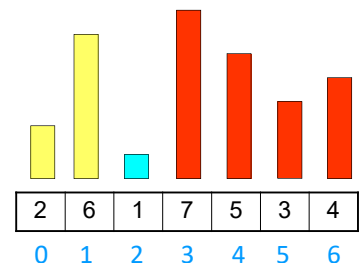


ソート済
位置

275

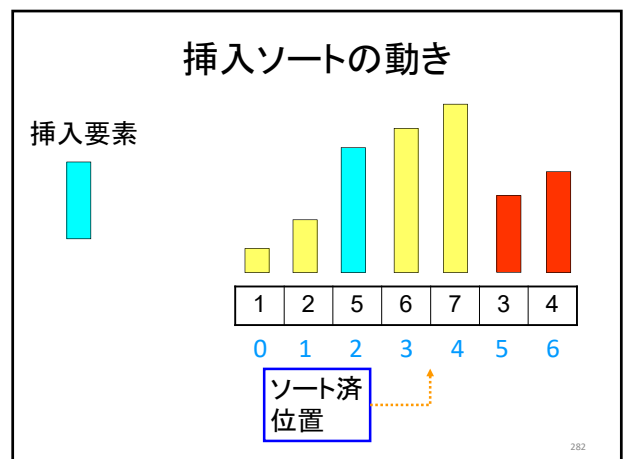
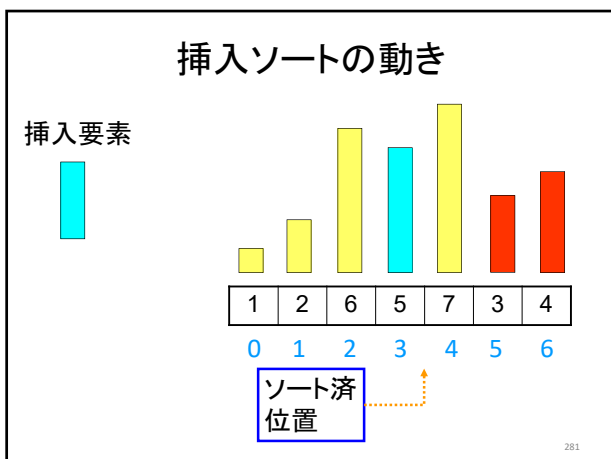
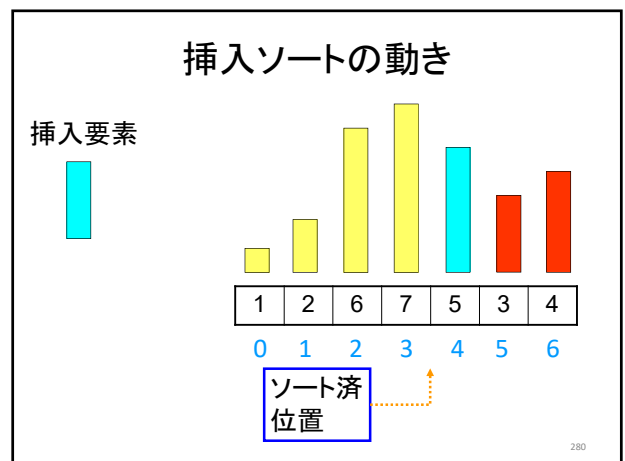
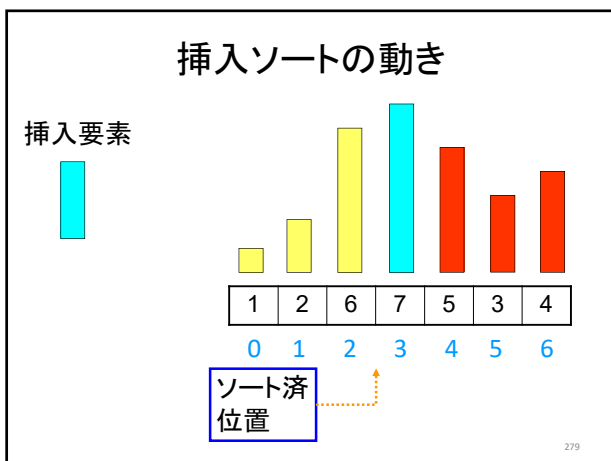
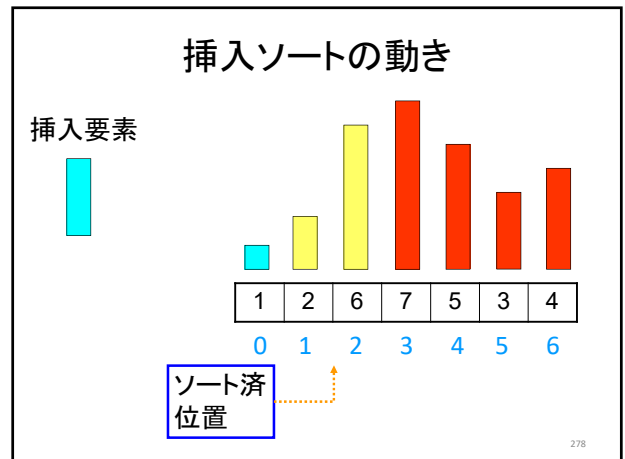
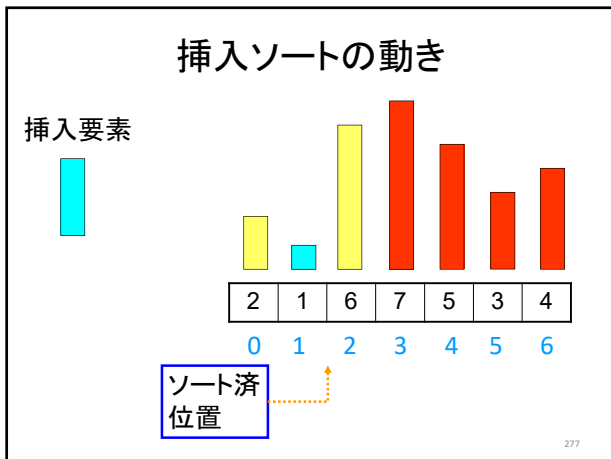
挿入ソートの動き

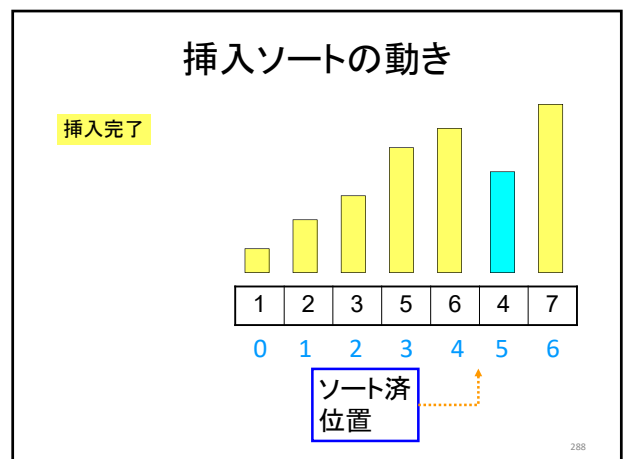
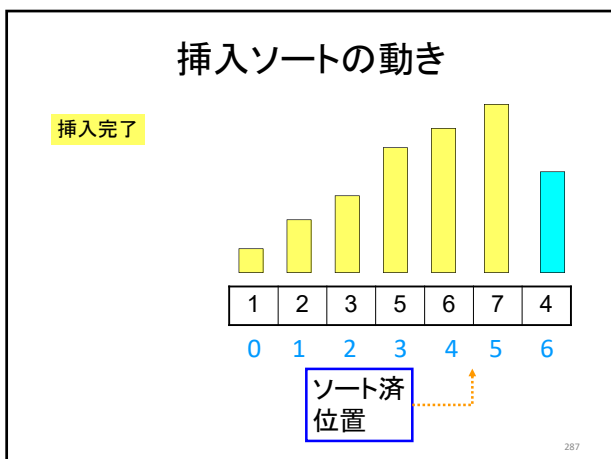
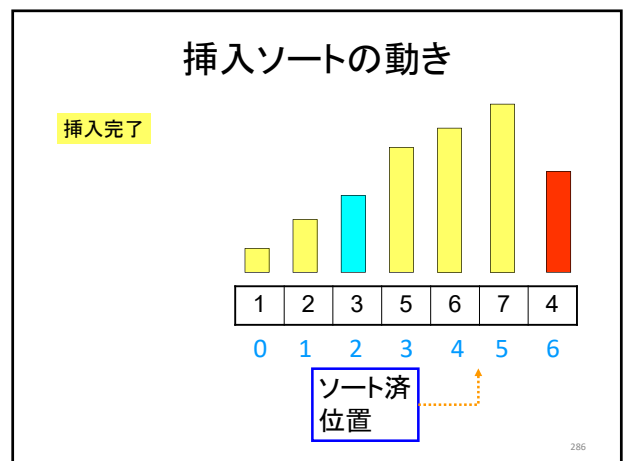
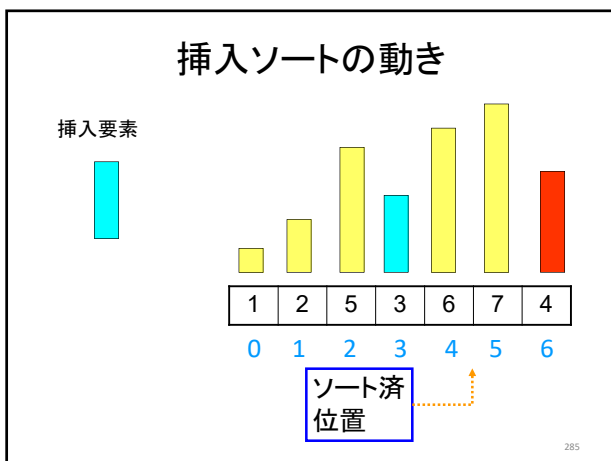
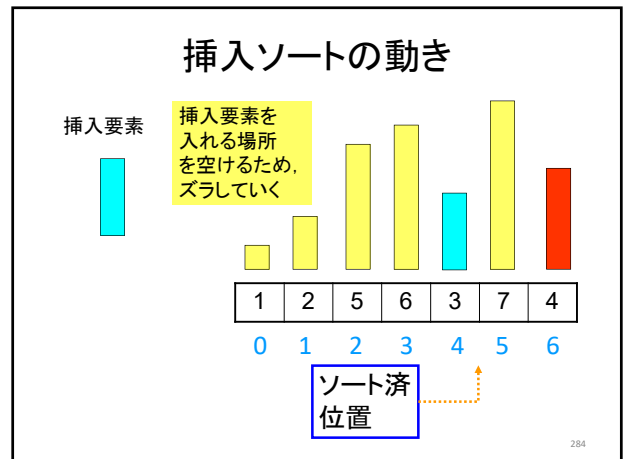
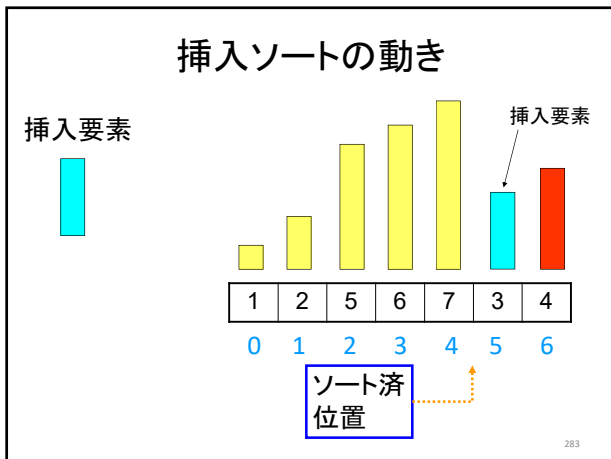
挿入要素



ソート済
位置

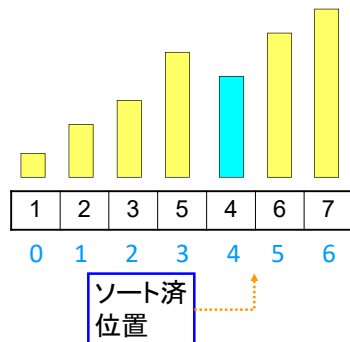
276





挿入ソートの動き

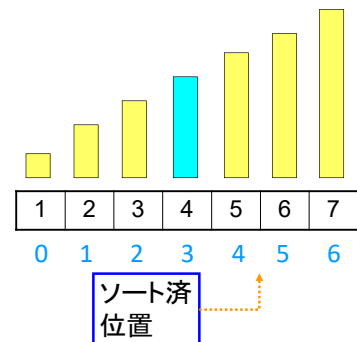
挿入完了



289

挿入ソートの動き

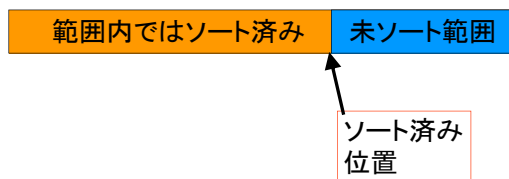
挿入完了



290

挿入ソートのポイント

- 挿入ソート: 途中でも、ソート済み位置の前はその範囲内での小さい順に並んでいる



291

挿入ソートの動きを理解しよう

- 挿入ソートをやってみよう!

292

挿入ソートのアルゴリズム(概略)

- 1: 未ソート部分について以下の処理を行う
 - 1-1: 未ソート部分の左端の値をソート済み部分に挿入する
- 2: ソート済み部分を増やす
(=未ソート部分を減らす)

293

挿入ソートのアルゴリズム(詳細)

1. ソート済み位置(s_pos)を、配列先頭の手前(-1)とする
2. ソート済み位置が配列末尾($N-1$)になるまで
ソート済み位置を1ずつ増やしなが、以下を繰り返す
 - 2-1. ソート済み位置+1の要素(s_pos+1)を挿入要素とする
 - 2-2. 挿入位置 ← ソート済み位置+1
 - 2-3. 現在位置 ← ソート済み位置
 - 2-4. 現在位置は配列先頭の要素番号以上であり、かつ現在位置の要素が挿入要素よりも大きい限り以下を繰り返す
 - 2-4-1. 現在位置の要素を1つ後ろの位置に格納する
 - 2-4-2. 挿入位置 ← 現在位置
 - 2-4-3. 現在位置を配列の先頭方向に1つ戻す
 - 2-5. 挿入要素を挿入位置に格納する

294

```
int data[N];      //要素数Nの配列
int s_pos;       // ソート済み位置
int i;           // 現在位置
```

コード例

```
for (s_pos = 1; s_pos < N; s_pos++){
    for (i = s_pos; i > 0; i--) {
        if (data[i-1] > data[i]) { // もし左が大きければ
            int t = data[i];       // 左に挿入(交換)
            data[i] = data[i-1];
            data[i-1] = t;
        }
    }
}
```

295

選択ソートのアルゴリズム(詳細)

0. ソートする配列をdata[N]とする
1. ソート済み位置(s_pos)を配列先頭(0)の手前(-1)とする
2. ソート済み位置(s_pos)が配列末尾(N-1)になるまで
ソート済み位置を1ずつ増やしながら以下を繰り返す
 - 2-1. ソート済み位置+1(s_pos + 1)を仮の最小値とし、
その位置(s_pos + 1)を最小位置(min_pos)格納
 - 2-2. 現在位置(i)をソート済み位置+2から
末尾まで変えながら、以下の処理を繰り返す
 - 2-2-1. 現在位置(i)の要素(data[i])が
最小位置の要素(data[min_pos])より小さいなら
その位置(i)を最小値(min_pos)に代入する
 - 2-3. 最小値とソート済み位置+1の要素を交換する

296

各種のソーティングアルゴリズム

バブルソート	$O(n^2)$	プログラムが単純
選択ソート	$O(n^2)$	データの交換回数が少ない 要素のデータ量が多い場合に有利
挿入ソート	$O(n^2)$	ほぼソートされた配列に対しては $O(1)$ で高速に動作
シェルソート	$O(n^{3/2})$ $\sim O(n^{7/6})$	実用的 理論的解析が困難
ヒープソート	$O(n \log n)$	実用的
クイックソート	$O(n \log n)$ ※ 平均値	有名だが、逆順ソートは $O(n^2)$ に近い効率

※ Nはデータ数

297

アルゴリズムの 計算量の求め方

計算量の考え方

データ数(n) $\rightarrow \infty$ (最大・最悪)のときの
計算量を、オーダーで考える

<データ数(n)における計算量の例>

小	$O(C)$	: 一定(データ数に無関係)
	$O(\log n)$	: nのlog : 二分探索
	$O(\sqrt{n})$	: nのルート(=0.5乗)
	$O(n)$	: n : 逐次探索
	$O(n^C)$	: nのC乗 : 1~nのC重ループ
	$O(c^n)$	: Cのn乗 : 1~Cのn重ループ
大	$O(n!)$	: nの階乗 : n個の総当たり

データ数最大時の計算量

データ数が少ない $\Rightarrow$ 計算量は問題ない
データ数が多い $\Rightarrow$ 計算量が問題になる

<データ数(n) $\rightarrow \infty$ の場合>

$$\begin{aligned}
 & \cdot n^3/10 + n + n^{0.5} = n^3 \\
 & \cdot 100 \times n^{0.1} + \log n = n^{0.1} \\
 & \cdot n \log n + n = n \log n \\
 & \cdot 2^n + n^{100} = 2^n \\
 & \cdot n! + 2^n = n!
 \end{aligned}$$

大きい方だけ
考えればよい

計算量を求めよう！その1

```
cin >> n;
for(i = 0 ; i < n ; i++){
    /*省略*/
}
for(j = 0 ; j < 20*n ; j++){
    /*省略*/
}
for(k = 0 ; k < 100 ; k++){
    /*省略*/
}
```

正解は・・・？

計算量を求めよう！その1

```
cin >> n;
for(i = 0 ; i < n ; i++){
    /*省略*/
}
for(j = 0 ; j < 20*n ; j++){
    /*省略*/
}
for(k = 0 ; k < 100 ; k++){
    /*省略*/
}
```

$O(n)$

$O(n)$

$O(C)$

最大は
 $O(n)$

計算量を求めよう！その2

```
cin >> n;
for(i = 0 ; i < n ; i+=10){
    for(j = 0 ; j < n ; j++){
        for(k = 0 ; k < 100 ; k++){
            /*省略*/
        }
    }
}
```

正解は・・・？

計算量を求めよう！その2

```
cin >> n;
for(i = 0 ; i < n ; i+=10){
    for(j = 0 ; j < n ; j++){
        for(k = 0 ; k < 100 ; k++){
            /*省略*/
        }
    }
}
```

$O(n \times n/10 \times 100)$

定数は無視: $O(n^2)$

計算量を求めよう！その3

```
void main(){
    cin >> n;
    X=Function(n)
}
int Function(int n){
    for(int i = 0 ; i < n ; i++){
        Function(n-1);
    }
}
```

正解は・・・？

計算量を求めよう！その3

```
void main(){
    cin >> n;
    X=Function(n)
}
int Function(int n){
    for(int i = 0 ; i < n ; i++){
        Function(n-1);
    }
}
```

$n \times n-1 \times n-2 \dots \Rightarrow O(n!)$

こんなコードを書かないこと！



クロージング テスト

今回、習ったことを復習するために、クロージングテストを行います

問題ごとに動画を止めて、答えをメモしましょう。

最後に、答えを動画で表示しますので、自分が理解しているか、確認しましょう。

```
int a;
switch (a){
    case 1:
        // 処理1
        break;
    case 2:
        // 処理2
        break;
    default:
        // 処理3
        break;
}
```

問1: 左のプログラムで, a=2 の場合に処理されるのは？

問2: 左のプログラムで, a=0 の場合に処理されるのは？

<選択肢>

- ① 処理1 ⑤ 処理2と3
- ② 処理2 ⑥ 処理1と3
- ③ 処理3 ⑦ 処理1と2と3
- ④ 処理1と2 ⑧ その他

クロージング テスト

【問3】

以下の選択肢を組み合わせて
if文の正しい表記を答えなさい

<選択肢>

- | | |
|-----------|-----------|
| ① {a==1} | ⑤ (b==2) |
| ② {a=1} | ⑥ (b=2) |
| ③ {a==1;} | ⑦ (b==2;) |
| ④ {a=1;} | ⑧ (b=2;) |
| | ⑨ if |
| | ⑩ その他, 空白 |

313

クロージング テスト

switch文の正しい表記を答えなさい
(aの値が1ならば, aに2を代入するプログラム)

【4】 case 1: 【5】

<選択肢>

- | | |
|----------|-----------|
| ① { | ⑥ a |
| ② } | ⑦ a=2 |
| ③ (| ⑧ a==2 |
| ④) | ⑨ ; |
| ⑤ switch | ⑩ その他, 空白 |

314

クロージング テスト

【問6】以下のelse文に対応するif文は？

- ```

① if(a == 1){
 ② if(b == 2){
 ③ if(c == 3){
 }
 }
 else{
 d = 4;
 }
}
④ その他

```

315

## 答え合わせ

全ての答えをメモしましたか？

## 解答

- |     |                           |
|-----|---------------------------|
| 【1】 | ②                         |
| 【2】 | ③                         |
| 【3】 | ⑨⑤④ if (b == 2) { a = 1;} |
| 【4】 | ⑤③⑥④① switch(a){          |
| 【5】 | ⑦⑨② a=2;}                 |
| 【6】 | ②                         |

お疲れさまでした

